

JUN – K

```
public class Test{
    public static void main(String[] args){
        A.staticX = 10;
        A a1 = new A();
        A a2 = new A();
        System.out.println(a2.x - a1.x);
        A a3 = a2;
        a3.x++;
        System.out.println(a2.x - a1.x);
    }
}
class A
{
    public int x;
    public static int staticX;
    public A(){staticX++; x = staticX;}
    public int getStaticX() {return staticX;}
}
```

Izlaz :

1

2

1. Koji su dozvoljeni potpisi defaultnog konstruktora ako je naziv klase Test ?

Test() / void Test(void) / **public Test()** / public Test(void) / public void Test()

2. Šta će biti rezultat izvršavanja sledećeg koda ? Obrazložiti :

```
class A{
    A(String message){
        System.out.println(message + " from A.");
    }
}
class B extends A{
    B(){
        System.out.println("Hello from B.");
    }
}
class RunSubClass{
    public static void main(String[] args){
        B b = new B();
    }
}
```

a) Hello from B

b) Hello from B

Hello from A

c) Hello from A

Hello from B

d) Kompajlerska greska na liniji 2

e) Kompajlerska greska na liniji 13

f) Greška prilikom izvršavanja.

- **Zato što konstruktor b poziva super() konstruktor a koji zahteva argument tipa String ali pošto mu B ne šalje ništa to će rezultovati greškom**

3. Šta predstavlja JVM, a Šta JRE ?

JVM(Java virtual machine)je interpreter koji izvršava Java bytecode a JRE(Java Runtime Environment) predstavlja okruženje koje je neophodno za izvršavanje Java programa. JRE sadrži u sebi JVM i neophodne biblioteke a JVM nam pruža mogućnost prenosivosti(prevod u byte code koji se nakon kompajliranja može izvršavati na JVM)

4. Neka je data nedovršena definicija klase

```
class Test{
    public static void main(String[] args){
        Rectangle r1 = new Rectangle(5,7);
        Rectangle r2 = new Rectangle(4,7);
        If(r1.equals(r2) == true){
            System.out.println("Isti su !!!"); }
        else { System.out.println("Nisu isti !!!");}
    }
}

class Recatngle{
    private int x,y;
    public Rectangle(int x,int y){
        this.x = x;
        this.x = x;
    }
    public boolean equals(Object o){
        if(o instanceof Rectangle){
            if(this.x == o.x && this.y == o.y){
                return true;
            }
            return false;
        }
    }
}
```

Definiši telo metoda equals i napiši primer upotrebe u poređenju stanja dva objekta tipa Rectangle

1. Objasniti sledeće pojmove

a. Učaurivanje

Učaurivanje ili enkapsulacija je mehanizam sakrivanja informacija pomoću modifikatora vidljivosti, postoje 4 tipa modifikatora vidljivosti a to su (public,protected,private i default)

b. Overloading

Overloading je mogućnost definisanja više metoda koje imaju isti naziv ali različit potpis npr različiti broj parametra ili različite vrste parametra, takođe je dozvoljeno da vraćaju različite tipove

c. Polimorfizam

Polimorfizam je mogućnost da varijablom određenog tipa referenciramo objekte različitih tipova I da automatski pozivamo metode koje su specifične za tip objekta koji varijabla referencira Uslovi za polimorfizam su :

- Poziv metoda potklase kroz varijablu bazne klase,
- Pozvana metoda mora biti član bazne klase
- Signatura metode I povratni tip moraju biti isti u baznoj I izvedenoj klasi
- Atribut pristupa ne sme biti restriktivniji u izvedenoj klasi nego što je u baznoj
- Polimorfizam se odnosi samo na metode

2. Konverzija složenih tipova u Javi

Postoje dve vrste konverzije u javi (iz užeg u širi i iz šireg u uži tip). Konverzija iz užeg u širi tip se vrši implicitno i nije potrebno castovanje dok je za konverziju iz šireg u uži potrebna eksplicitna konverzija koja se vrši castvanjem (**Primer gradjanin i student**)
`Student s1 = new Student(); // Gradnjanin g = s1` sada s1 može koristiti metode I promenljive iz klase gradjanin kod ako želimo da g koristi metode iz studenata potrebno je izvršiti cast `(Student)(g)`

3. Final modifikator

Final je modifikator koji može stajati uz promenljivu, metodu ili klasu

Ukoliko stoji uz promenljivu ta promenljiva može dobiti vrednost samo jednom nakon čega se ne može menjati, ako stoji uz metodu tada se ta metoda ne može prepisati a ako stoji uz klasu tada se ta klasa ne može naslediti

4. Anonimne klase (navesti primer)

Anonimna klasa je modifikacija neke već postojeće klase I ona se kreira u delu koda gde se poziva instanciranje neke već postojeće klase(dodavanjem vitičastih zagrada nakon poziva konstruktora) naziva se anonimna zato što joj ne znamo ime I ne možemo je ponovo pozvati.

```
A a = new A(){  
    /*modifikacija klase A*/  
}
```

5. Šta su niti? Kako se Javi definišu? Šta biva zaključano pozivom sinhronizovanog metoda nekog objekta ? Šta se dešava kada unutar sinhronizovanog metoda bude pozvan wait()?

Niti su skup koraka koji se izvršavaju sekvencijalno, niti predstavljaju deo procesa I u jednom programu možemo imati više niti te niti se mogu izvršavati redom ili paralelno. U javi su definisani u klasi Thread I definišu se sa

```
Thread nit = new Thread();  
A pokreću se sa komandom nit.start();
```

Kada se pokrene komanda start to pokreće metodu run tamo gde je ona implementirana. Drugi način kreiranja niti jeste pomoću implementacije interfejsa runnable. Pozivom sinhronizovanog metoda omogućava se ekskluzivan pristup podacima za nit koja je u svom kritičnom regionu .

Pozivom metode wait se pauzira rad za odredjenu nit dok ne budu zadovoljeni odredjeni uslovi

1. Dopuni main metod tako da bude izvršen metod printit(), a zatim ispiši šta će biti rezultat izvršavanja programa.

```
public class InitialTest{
    int x;
    public static void main(String[] args){
        InitialTest t = new InitialTest();
        t.printIt();
    }
    public void printIt(){
        int y = 2;
        System.out.println(x + " " + y);
    }
}
```

Izlaz : **0 2**

2. Razlika između statičkih i nestatičkih metoda.

Statičke metode ne mogu da pristupe nestatičkim podacima iz razloga što ne mogu da koriste parametar this, dok nestatičke metode mogu da koriste i statičke i nestatičke podatke

3. Neka su definisani sledeći tipovi :

```
public interface Merljivo(){
    double getVelicina();
}
public class Kvadrat implements Merljivo{
    double x,y;
    public Kvadrat(double x1,double y1){
        x = x1;
        y = y1;
    }
}
```

Koje od sledećih linija koda će biti prihvaćene od strane kompajlera

- a) **merljivo m = new Kvadrat(7,7);**
- b) **Kvadrat m = new Kvadrat(7,7);**
- c) merljivo m = new merljivo(7,7);
- d) merljivo m = new merljivo();
- e) merljivo m = new Kvadrat();
- f) Kvadrat m = new Kvadrat();

4. Šta je Garbage collector i čemu služi

Garbage collector je nit koja vodi računa o tome da briše sve one objekte na koje ne pokazuje nijedna referenca, ona to radi kada je potrebno osloboditi memoriju ili kada to dozvoli procesorsko vreme

5. Imajući u vidu naredni kod šta je od navedenog tačno

```
class A{  
    A(){ }  
}  
class B extends A{  
}
```

- a) Konstruktor klase B je javni.
- b) Konstruktor klase B nema argumente.**
- c) Konstruktor klase B uključuje poziv this().
- d) Konstruktor klase B uključuje poziv super().**
- e) Konstruktor klase B ne postoji.
- f) Greška u kompajliranju

1. Objasniti sledeće pojmove

a. Učaurivanje

Učaurivanje ili enkapsulacija je mehanizam sakrivanja informacija pomoću modifikatora vidljivosti, postoje 4 tipa modifikatora vidljivosti a to su (public,protected,private i default)

b. Polimorfizam

Polimorfizam je mogućnost da varijablom određenog tipa referenciramo objekte različitih tipova i da automatski pozivamo metode koje su specifične za tip objekta koji varijabla referencira Uslovi za polimorfizam su :

- Poziv metoda potklase kroz varijablu bazne klase,
- Pozvana metoda mora biti član bazne klase
- Signatura metode i povratni tip moraju biti isti u baznoj i izvedenoj klasi
- Atribut pristupa ne sme biti restriktivniji u izvedenoj klasi nego što je u baznoj
- Polimorfizam se odnosi samo na metode

c. Prenosivost (na šta se odnosi i kako je realizovana u Javi)

Java kod se prevodi u bytecode koji nije mašinski jezik nijednog konkretnog računara već se izvršava na JVM tako da je za kompajlovanje java kod potrebna samo JVM

2. Konverzija složenih tipova u Javi.

Postoje dve vrste konverzije u javi (iz užeg u širi i iz šireg u užu tip). Konverzija iz užeg u širi tip se vrši implicitno i nije potrebno castovanje dok je za konverziju iz šireg u užu potrebna eksplicitna konverzija koja se vrši castvanjem (Primer gradjanin i student)

Student s1 = new Student(); // Gradnjanin g = s1 sada s1 može koristiti metode i promenljive iz klase gradjanin kod ako želimo da g koristi metode iz studenata potrebno je izvršiti cast (Student)(g)

3. Base i Super. Šta predstavljaju i kako se mogu koristiti ?

Base ne postoji u javi ali postoji *this* to je eksplicitni parameter kojim možemo pristupiti podacima iz klase u kojoj se trenutno nalazimo *this()* je poziv defaultnog konstruktora klase, *super* je eksplicitni parameter kojim se pristupa podacima iz izvedene/naslednjene klase *super()* je poziv konstruktora iz nasleđene klase

Proveravani izuzeci su izuzeci izvedeni iz klase Exception I oni nisu izvedeni iz klase RuntimeException. Ako metoda baca neki proverevani izuzetak poziv te metode mora biti uokviren try/catch blokom a metoda mora da bude označena sa throws I nazivom klase koji izuzetak baca

Neproveravani izuzeci su izuzeci koji su izvedeni iz klase RuntimeException poziv metode koje baca ovakav izuzetak, Može !!!

4. Šta su niti? Kako se Javi definišu? Šta biva zaključano pozivom sinhronizovanog metoda nekog objekta ? Šta se dešava kada unutar sinhronizovanog metoda bude pozvan wait()?

Niti su skup koraka koji se izvršavaju sekvencijalno, niti predstavljaju deo procesa I u jednom delu programa možemo imati više niti te niti se mogu izvršavati redom ili paralelno. U javi su definisani u klasi Thread I definišu se sa

```
Thread nit = new Thread();
```

```
A pokreću se sa komandom nit.start();
```

Kada se pokrene komanda start to pokreće metodu run tamo gde je ona implementirana. Drugi način kreiranja niti jeste pomoću implementacije interfejsa runnable. Pozivom metoda sinhronizovanog metoda omogućava se ekskluzivan pristup podacima za nit koja je u svom kritičnom regionu .

Pozivom metode wait se pauzira rad za određena nit dok ne budu zadovoljeni određeni uslovi

AVG – K

1. Koja je podrazmevana vidljivost podataka definisanih u klasi , a koja podataka definisanih u interfejsu

U klasi default(package friendly) dok je u interface public

2. Napisati implementaciju klase Vektor (celobrojne coordinate u dekartovom sistemu) koju je moguće upotrebiti na način dat u klasi Test

```
class Test{
    public static void main(String[] args){
        vector t = new Vektor(1,1);
        vector t = new Vektor(1,1);
        int v = t.skalarmiProizvod(t2);
    }
}
class Vektor{
    int x,y;
    public Vektor(int x,int y){
        this.x = x;
        this.y = y;
    }

    int skalarmiProizvod(Vektor t2){
        if(t2 instanceof Vektor){
            return this.x*t2.x + this.y*t2.y
        }
    }
}
```

3. Zašto statički metodi ne mogu pristupiti nestatičkim podacima definisanim u klasi u kojoj su sami definisani

Zato što nemaju mogućnost korišćenja operatora this

4. Da li apstraktna klasa može implementirati interfejs

Može

5. Šta će biti rezultat izvršavanja sledećeg koda

public class Test{

```
    public static void main(String[] args){
        A.staticX = 10;
        A a1 = new A();
        A a2 = a1;
        a2.x+=10;
        System.out.println(a2.x - a1.x);
        a2 = new A();
        System.out.println(a2.x - a1.x);
    }
}
```

class A

```
{
    public int x;
    public static int staticX;
    public A(){staticX++; x=staticX;}
    public int getStaticX(){return staticX;}
}
```

Izlaz : **0 -9**

AVG – A

1. Polimorfizam – definicija uslovi, Dati primer

Polimorfizam je mogućnost da varijablom određenog tipa referenciramo objekte različitih tipova i da automatski pozivamo metode koje su specifične za tip objekta koji varijabla referencira Uslovi za polimorfizam su :

- Poziv metoda potklase kroz varijablu bazne klase,
- Pozvana metoda mora biti član bazne klase
- Signatura metode i povratni tip moraju biti isti u baznoj i izvedenoj klasi
- Atribut pristupa ne sme biti restriktivniji u izvedenoj klasi nego što je u baznoj
- Polimorfizam se odnosi samo na metode

Primer: Možemo imati klasu *geometrijskiOblik* i klase *kvadrat* i *pravougaonik* klase koje nasleđuju klasu *geometrijskiOblik* mi zatim možemo kreirati niz geometrijskihOblika gde mogu biti smešteni i kvadrati i pravougaonici

```
geometrijskiOblik g1 = new Kvadrat(); geometrijskiOblik g2 = new Pravougaonik();
```

2. Anonimne klase, Dati primer.

Anonimna klasa je modifikacija neke već postojeće klase I ona se kreira u delu koda gde se poziva instanciranje neke već postojeće klase (dodavanjem vitičastih zagrada nakon poziva konstruktora) naziva se anonimna zato što joj ne znamo ime I ne možemo je ponovo pozvati.

```
A a = new A(){  
    /*modifikacija klase A*/  
}
```

3. Šta je eksplicitna konverzija I kada ju je potrebno primeniti kada su referencni tipovi u pitanju, Dati primer.

Postoje dve vrste konverzije u javi (iz užeg u širi i iz šireg u užu tip). Konverzija iz užeg u širi tip se vrši implicitno i nije potrebno castovanje dok je za konverziju iz šireg u užu potrebna eksplicitna konverzija koja se vrši castvanjem (Primer gradjanin i student)

Student s1 = new Student(); // Gradnjanin g = s1 sada s1 može koristiti metode I promenljive iz klase gradjanin kod ako želimo da g koristi metode iz studenata potrebno je izvršiti cast (Student)(g)

4. Konstruktori, Dati primer.

Konstruktor je posebna vrsta metode koja mora do poseduje sledece karakteristike

- Mora se nazivati isto kao I klasa gde je definisan
- Ne sme imati povratnu vrednost
- Poziva se isključivo sa instanciranjem (operatorom new)

Neka klasa može imati više konstruktora a ukoliko se ne definiše nijedan konstruktor tada klasa poseduje defaultni konstruktor (klasa(){}).

5. Da li se u dete klasi može naći atribut koji je istog tipa, naziva I vidljivosti kao I atribut nasleđen od roditelja . Ukoliko je moguće, kako je moguće pristupiti nasleđenom podatku.

Može I ukoliko se desi takav slučaj možemo pristupiti podatku iz roditelja pomoću eksplicitnog parametra super (super.promenljiva nam daje promenljivu iz roditeljske klase)

1. Napisati definiciju klase koja sadrži :

- a. dve privatne promenljive celobrojnog tipa x i y**
- b. konstruktor koji prima vrednosti na koje setuje x i y**
- c. metod equals koji poredi stanje tekućeg objekta sa stanjem objekta koji dobija u argumentu.**

Napisati testnu klasu u kojoj se kreiraju dva različita objekta prethodno definisane klase inicijalizovana istim vrednostima, a zatim ih poredi po stanju , pa po referenci i ispisuje odgovarajuću poruku.

```
class Tacka {  
    private int x; private int y;  
    public Tacka(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    public boolean equals(Object o) {  
        if (o instanceof Tacka)  
        {  
            Tacka t = (Tacka)o;  
            if (t.x==x && t.y==y)  
                return true;  
        }  
        return false;  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Tacka t1 = new Tacka(3,5);  
        Tacka t2 = new Tacka(3,5);  
        if (t1.equals(t2))  
            System.out.println("Ista stanja");  
        else  
            System.out.println("Razlicita stanja");  
  
        if (t1==t2) System.out.println("Iste reference");  
        else System.out.println("Razlicite reference");  
    }  
}
```

2. Razlika između statičkih i nestatičkih metoda.

Statičke metode ne mogu da pristupe ne statičkim podacima iz razloga što ne mogu da koriste this, dok nestatičke metode mogu da koriste i nestatičke i statičke podatke

3. Šta je Garbage collector i čemu služi

Garbage collector je nit koja vodi računa o tome da briše sve one objekte na koje ne pokazuje nijedna referenca, ona to radi kada je potrebno osloboditi memoriju ili kada to dozvoli procesorsko vreme

4. Po čemu se razlikuju podaci članovi klase i objekata

Podaci klase su podaci koji su zajednički za sve objekte te klase imaju ključnu reč static pored sebe, takodje statičke metode ne mogu da koriste ne statičke podatke iz razloga što ne poseduju parametra this, dok su podaci objekta zasebni podaci za svaki objekat to su nestatički podaci i oni mogu da koriste i statičke i nestatičke metode

SEP – A

1. Na šta se odnose pojmovi prenosivost i ponovnog korišćenja koda (reusability) i kojim mehanizmom su obezbeđeni u Javi

Na prenosivost se odnosi to da se Java kod prevodi u bYTE kod koji nije mašinski kod za nijedan konkretan računar već je to bYTE kod koji se izvršava u JVM (Java Virtual Machine) što znači da kompajlirani kod mogu da izvršavaju više JVM-a

2. Konverzija složenih tipova u Javi

Postoje dve vrste konverzije u Javi (iz užeg u širi i iz šireg u uži tip). Konverzija iz užeg u širi tip se vrši implicitno i nije potrebno castovanje dok je za konverziju iz šireg u uži potrebna eksplicitna konverzija koja se vrši castvanjem (Primer gradjanin i student)

Student s1 = new Student(); // Gradnjanin g = s1 sada s1 može koristiti metode i promenljive iz klase gradjanin kod ako želimo da g koristi metode iz studenata potrebno je izvršiti cast (Student)(g)

3. Enkapsulacija – definisati i navesti način izvođenja enkapsulacije u Javi.

Učaurivanje ili enkapsulacija je mehanizam sakrivanja informacija pomoću modifikatora vidljivosti, postoje 4 tipa modifikatora vidljivosti a to su (public,protected,private i default)

4. Statičko I dinamičko vezivanje u Javi. Koje se I gde primenjuje

5. Šta su niti? Kako se Javi definišu? Šta biva zaključano pozivom sinhronizovanog metoda nekog objekta ? Šta se dešava kada unutar sinhronizovanog metoda bude pozvan wait()?

Niti su skup koraka koji se izvršavaju sekvencijalno, niti predstavljaju deo procesa I u jednom delu programa možemo imati više niti te niti se mogu izvršavati redom ili paralelno. U javi su definisani u klasi Thread I definišu se sa

```
Thread nit = new Thread();
```

```
A pokreću se sa komandom nit.start();
```

Kada se pokrene komanda start to pokreće metodu run tamo gde je ona implementirana. Drugi način kreiranja niti jeste pomoću implementacije interfejsa runnable. Pozivom metoda sinhronizovanog metoda omogućava se ekskluzivan pristup podacima za nit koja je u svom kritičnom regionu .

Pozivom metode wait se pauzira rad za odredjeni process dok ne budu zadovoljeni odredjeni uslovi